

환경 및 소코핑의 이해

R 프로그래밍 with workflow design



환경과 스코핑의 이해

왜 환경과 스코핑을 이해해야 할까?

- R을 단순한 분석 도구가 아니라 **프로그래밍 언어**로 활용하기 위해 꼭 필요한 개념임
- 함수나 객체가 어디에 저장되고 어떻게 검색되는지 이해해야 오류를 줄일 수 있음
- 여러 패키지를 사용하는 경우 이름 충돌과 함수 마스킹 문제가 자주 발생함
- 복잡한 분석 흐름이나 자동화된 코드, 사용자 함수 작성을 할 때 환경 구조를 모르면 구현이 어려움
- 환경과 스코핑은 R 초급 사용자와 중급 사용자를 구분 짓는 핵심 개념

환경과 검색 경로의 기초

- R의 모든 객체(데이터, 함수 등)는 **특정 환경(environment)**에 저장됨
- `Environment` 탭: 현재 세션의 객체와 환경 확인 가능
- **전역 환경(Global Environment)**이 기본으로 표시됨
- 불러온 패키지들의 환경도 함께 확인 가능

R의 환경 구조와 검색 순서

- 환경은 `R_GlobalEnv` → `R_EmptyEnv` 방향의 트리 구조로 연결됨
- 패키지를 불러오면 해당 패키지의 환경이 검색 경로의 최상단에 추가됨
- 최근에 불러온 패키지일수록 먼저 검색되어 함수가 우선 적용됨
- 동일한 이름의 함수가 여러 패키지에 있을 경우 나중에 불러온 패키지가 우선
- `search()` 함수를 사용하면 현재 세션의 검색 경로 순서를 확인할 수 있음
- 이 순서는 객체나 함수를 찾을 때 R이 따르는 탐색 기준이 됨

pryr::parenvs() 함수

- pryr 패키지의 parenvs() 함수를 사용하면 **환경의 전체 구조(트리 형태)**를 쉽게 파악할 수 있음
  - e: 대상 환경 또는 객체를 인자로 받음 (지정하지 않으면 현재 환경 사용)
  - `all`: TRUE일 경우 모든 부모 환경까지 출력

```
library(pryr)
parenvs(all = TRUE)
```

```
##    label                                name
## 1  <environment: R_GlobalEnv>           ""
## 2  <environment: package:pryr>          "package:pryr"
## 3  <environment: package:stats>         "package:stats"
## 4  <environment: package:graphics>      "package:graphics"
## 5  <environment: package:grDevices>     "package:grDevices"
## 6  <environment: package:utils>         "package:utils"
## 7  <environment: package:datasets>      "package:datasets"
## 8  <environment: package:methods>       "package:methods"
## 9  <environment: 0x0000019ba71462a0>    "Autoloads"
## 10 <environment: base>                  ""
## 11 <environment: R_EmptyEnv>            ""
```

함수 마스킹과 검색 경로

- `library(dplyr)`를 실행하면 아래와 같은 메시지가 출력됨:

```
##
## Attaching package: 'dplyr'
## The following objects are masked from 'package:stats':
##
##     filter, lag
        ...
```

- 이는 `dplyr` 패키지의 함수가 `stats` 패키지의 동일한 이름의 함수보다 먼저 검색되었기 때문
- 나중에 불러온 `dplyr`이 검색 경로에서 더 앞에 위치해 있어서 `stats::filter()`가 아닌 `dplyr::filter()`가 호출됨
- 현재 검색 순서는 `search()`로 확인 가능:

```
search()
```

```
##  [1] ".GlobalEnv"        "package:pryr"      "package:stats"
##  [4] "package:graphics"  "package:grDevices" "package:utils"
##  [7] "package:datasets"  "package:methods"   "Autoloads"
## [10] "package:base"
```

런타임 환경

런타임 환경이란?

- 함수가 호출될 때마다 실행을 위한 임시 환경이 생성되는데, 이를 **런타임 환경(**runtime environment**)**이라고 부름
- 함수 내에서 생성된 객체는 런타임 환경에 저장됨
- 함수 실행이 끝나면 이 환경은 사라지며 객체도 함께 제거됨
- 따라서 전역 환경의 객체와 이름이 같더라도 별개의 객체임

런타임 환경의 구조 확인

```
add_numbers = function() {
  nums = 1:10
  sum_nums = sum(nums) # nums, sum_nums는 런타임 환경에만 존재함
  return(list(
    value = sum_nums,
    runtime_env = environment(), # 함수가 실행된 환경
    parent_env = parent.env(environment()), # 런타임 환경의 부모
    objects_env = ls.str(environment()) # 런타임 환경에 존재하는 객체 목록
  ))
}
```

전역 환경과 독립적인 런타임 환경

```
nums = 1:5
add_numbers()
```

```
## $value
## [1] 55
##
## $runtime_env
## <environment: 0x0000019bb030e580>
##
## $parent_env
## <environment: R_GlobalEnv>
##
## $objects_env
## nums :  int [1:10] 1 2 3 4 5 6 7 8 9 10
## sum_nums :  int 55
```

```
print(nums)  # 전역 환경의 nums는 여전히 1:5
```

```
## [1] 1 2 3 4 5
```

부모 환경을 참조하는 런타임 환경

```
add_numbers_2 = function() {
  sum(nums)   # nums가 내부에 없을 때
}
nums = 1:5
add_numbers_2()
```

```
## [1] 15
```

- 함수 내부에 nums가 없으면 스코핑 규칙에 따라 **부모 환경(전역 환경)**에서 객체를 찾음
- 전역 환경에 nums = 1:5가 있으므로 합계 15를 반환함
- 이는 런타임 환경이 연결된 상위 환경을 따라 탐색하기 때문

안전한 함수 작성 방식

```
nums = 1:5
modify_nums = function(nums) {
  nums = nums + 1
  return(nums)
}
nums = modify_nums(nums)
```

- 인자를 명시적으로 함수에 전달함
- 반환값을 전역 환경에 명시적으로 재할당함
- 이렇게 하면 전역 환경과 불필요한 충돌 없이 안전한 수정이 가능함
- 초보자가 흔히 겪는 런타임-전역 혼동을 방지할 수 있음